ENDLESS RUNNER GAME

DEVELOPMENT USING PYTHON TKINTER

AIM

To design and develop an interactive 2D endless-running game using Python Tkinter. The player moves a runner between three lanes, jumps over obstacles, collects coins, increases the score, and tries to survive for as long as possible. The game includes keyboard controls, animation, collision detection, score tracking, restart, and exit options.

PROCEDURE

1. Import the required Python libraries: tkinter and random.

2. Define the game constants: window width and height, lane positions, player position, and the timer interval.

3. Create the main Tkinter window and a Canvas for the game scene.

4. Add the title label, the controls label, and the score and coins label above the canvas.

5. Add the RESTART GAME and EXIT buttons below the canvas.

6. Draw a road with three lanes, side scenery, a runner, coins, and obstacles.

7. Use the left and right arrow keys to change lanes.

8. Use the up arrow key or space bar to jump.

9. Bind the keyboard events to their handler functions using root.bind().

10. Generate coins and obstacles at random lanes and move them downward to simulate forward running.

11. Increase the score over time and award bonus points for collecting coins.

12. Detect collisions with obstacles; jumping can avoid an obstacle during the jump period.

13. Gradually increase the game speed to make the game more challenging.

14. Redraw the whole scene on every frame and schedule the next update using the after() method.

15. Display GAME OVER with the final score and provide Restart Game and Exit buttons.

16. On restart, cancel the running timer and reset the lane, score, coins, speed, and objects.

17. Save the program as endless_runner_game.py and run it using Python on a desktop with Tkinter installed.

18. Play the game, capture screenshots of the running window, and verify the output.

PROGRAM CODE

Save the following program as **endless_runner_game.py**.

```
import tkinter as tk
import random

WIDTH = 600
HEIGHT = 650
LANES = [150, 300, 450]
PLAYER_Y = 535
TICK_MS = 35

class EndlessRunner:
    def __init__(self, root):
        self.root = root
        self.root.title("ENDLESS RUNNER")
        self.root.resizable(False, False)
        self.root.configure(bg="#101820")

        tk.Label(
            root, text="ENDLESS RUNNER",
            font=("Arial", 22, "bold"),
            bg="#101820", fg="#FFD166"
        ).pack(pady=(10, 2))
        tk.Label(
            root,
            text="LEFT/RIGHT: change lane   UP: jump   Collect coins and dodge obstacles",
            font=("Arial", 9),
            bg="#101820", fg="white"
        ).pack(pady=(0, 6))

        self.info = tk.Label(
            root, text="SCORE: 0   COINS: 0",
            font=("Arial", 12, "bold"),
            bg="#101820", fg="#7AE7C7"
        )
        self.info.pack(pady=4)

        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT,
            bg="#243447", highlightthickness=2,
            highlightbackground="#FFD166"
        )
        self.canvas.pack(padx=12, pady=5)

        buttons = tk.Frame(root, bg="#101820")
        buttons.pack(pady=(3, 12))
        tk.Button(
            buttons, text="RESTART GAME", command=self.restart,
            font=("Arial", 10, "bold"), bg="#7AE7C7", padx=10
        ).pack(side="left", padx=6)
        tk.Button(
            buttons, text="EXIT", command=root.destroy,
            font=("Arial", 10, "bold"), bg="#FF6B6B", padx=18
        ).pack(side="left", padx=6)

        root.bind("<Left>", self.move_left)
        root.bind("<Right>", self.move_right)
        root.bind("<Up>", self.jump)
        root.bind("<space>", self.jump)

        self.timer = None
        self.restart()

    def restart(self):
        if self.timer is not None:
            try:
                self.root.after_cancel(self.timer)
            except tk.TclError:
                pass
        self.lane = 1
        self.player_y = PLAYER_Y
        self.jump_frames = 0
        self.score = 0
        self.coins = 0
        self.speed = 7
        self.objects = []
        self.spawn_counter = 0
        self.running = True
        self.info.config(text="SCORE: 0   COINS: 0")
        self.draw_scene()
        self.update_game()

    def move_left(self, event=None):
        if self.running and self.lane > 0:
            self.lane -= 1
            self.draw_scene()

    def move_right(self, event=None):
        if self.running and self.lane < 2:
            self.lane += 1
            self.draw_scene()

    def jump(self, event=None):
        if self.running and self.jump_frames == 0:
            self.jump_frames = 16
```

PROGRAM CODE (CONTINUED)

```
    def spawn_object(self):
        lane = random.randrange(3)
        kind = random.choice(["coin", "coin", "obstacle"])
        self.objects.append({"lane": lane, "y": -30, "kind": kind})

    def draw_road(self):
        # Side scenery
        self.canvas.create_rectangle(0, 0, 70, HEIGHT, fill="#1C6E4A", outline="")
        self.canvas.create_rectangle(530, 0, WIDTH, HEIGHT, fill="#1C6E4A", outline="")
        # Road
        self.canvas.create_rectangle(70, 0, 530, HEIGHT, fill="#394653", outline="")
        # Lane markers
        for y in range(-20, HEIGHT, 80):
            self.canvas.create_rectangle(222, y, 228, y + 42, fill="#DDE3E8", outline="")
            self.canvas.create_rectangle(372, y, 378, y + 42, fill="#DDE3E8", outline="")
        # Simple trees / scenery
        for y in range(35, HEIGHT, 125):
            self.canvas.create_oval(15, y, 55, y + 40, fill="#2EA66A", outline="")
            self.canvas.create_rectangle(32, y + 28, 39, y + 55, fill="#7A4A2A", outline="")
            self.canvas.create_oval(545, y + 30, 585, y + 70, fill="#2EA66A", outline="")
            self.canvas.create_rectangle(562, y + 58, 569, y + 85, fill="#7A4A2A", outline="")

    def draw_player(self, x, y):
        # Cartoon runner
        self.canvas.create_oval(x - 20, y - 58, x + 20, y - 18,
                                fill="#FFD6A5", outline="#17212B", width=2)
        self.canvas.create_oval(x - 22, y - 20, x + 22, y + 38,
                                fill="#06D6A0", outline="#17212B", width=2)
        self.canvas.create_line(x - 12, y + 35, x - 20, y + 52,
                                fill="#FFD166", width=7)
        self.canvas.create_line(x + 12, y + 35, x + 20, y + 52,
                                fill="#FFD166", width=7)
        self.canvas.create_line(x - 18, y - 8, x - 33, y + 8,
                                fill="#FFD6A5", width=6)
        self.canvas.create_line(x + 18, y - 8, x + 33, y + 4,
                                fill="#FFD6A5", width=6)
        # Eyes
        self.canvas.create_oval(x - 9, y - 45, x - 4, y - 40, fill="#17212B", outline="")
        self.canvas.create_oval(x + 4, y - 45, x + 9, y - 40, fill="#17212B", outline="")

    def draw_scene(self):
        self.canvas.delete("all")
        self.draw_road()
        # Draw objects
        for obj in self.objects:
            x = LANES[obj["lane"]]
            y = obj["y"]
            if obj["kind"] == "coin":
                self.canvas.create_oval(x - 13, y - 13, x + 13, y + 13,
                                        fill="#FFD166", outline="#FFF0A6", width=3)
                self.canvas.create_text(x, y, text="$", font=("Arial", 13, "bold"),
                                        fill="#8A5A00")
            else:
                self.canvas.create_rectangle(x - 28, y - 22, x + 28, y + 22,
                                             fill="#EF476F", outline="#FFD6DE", width=2)
                self.canvas.create_text(x, y, text="!", font=("Arial", 20, "bold"),
                                        fill="white")
        player_y = self.player_y - (70 if self.jump_frames > 0 else 0)
        self.draw_player(LANES[self.lane], player_y)
        self.canvas.create_text(18, 18, anchor="nw", text="RUN!",
                                font=("Arial", 12, "bold"), fill="#FFD166")
        if not self.running:
            self.canvas.create_rectangle(105, 235, 495, 390,
                                         fill="#101820", outline="#FFD166", width=3)
            self.canvas.create_text(WIDTH // 2, 275, text="GAME OVER",
                                    font=("Arial", 28, "bold"), fill="#FF6B6B")
            self.canvas.create_text(WIDTH // 2, 320,
                                    text=f"Score: {self.score}   Coins: {self.coins}",
                                    font=("Arial", 16, "bold"), fill="white")
            self.canvas.create_text(WIDTH // 2, 355,
                                    text="Press RESTART GAME to try again",
                                    font=("Arial", 11), fill="#7AE7C7")

    def update_game(self):
        if not self.running:
            return

        self.score += 1
        self.spawn_counter += 1
        if self.spawn_counter >= max(10, int(28 - self.speed)):
            self.spawn_object()
            self.spawn_counter = 0

        if self.jump_frames > 0:
            self.jump_frames -= 1

        player_hit_y = self.player_y
        remaining = []
        for obj in self.objects:
            obj["y"] += self.speed
            same_lane = obj["lane"] == self.lane
            close_y = abs(obj["y"] - player_hit_y) < 42
```

PROGRAM CODE (CONTINUED)

```
            if same_lane and close_y:
                if obj["kind"] == "coin":
                    self.coins += 1
                    self.score += 25
                    continue
                elif self.jump_frames == 0:
                    self.running = False
                    break

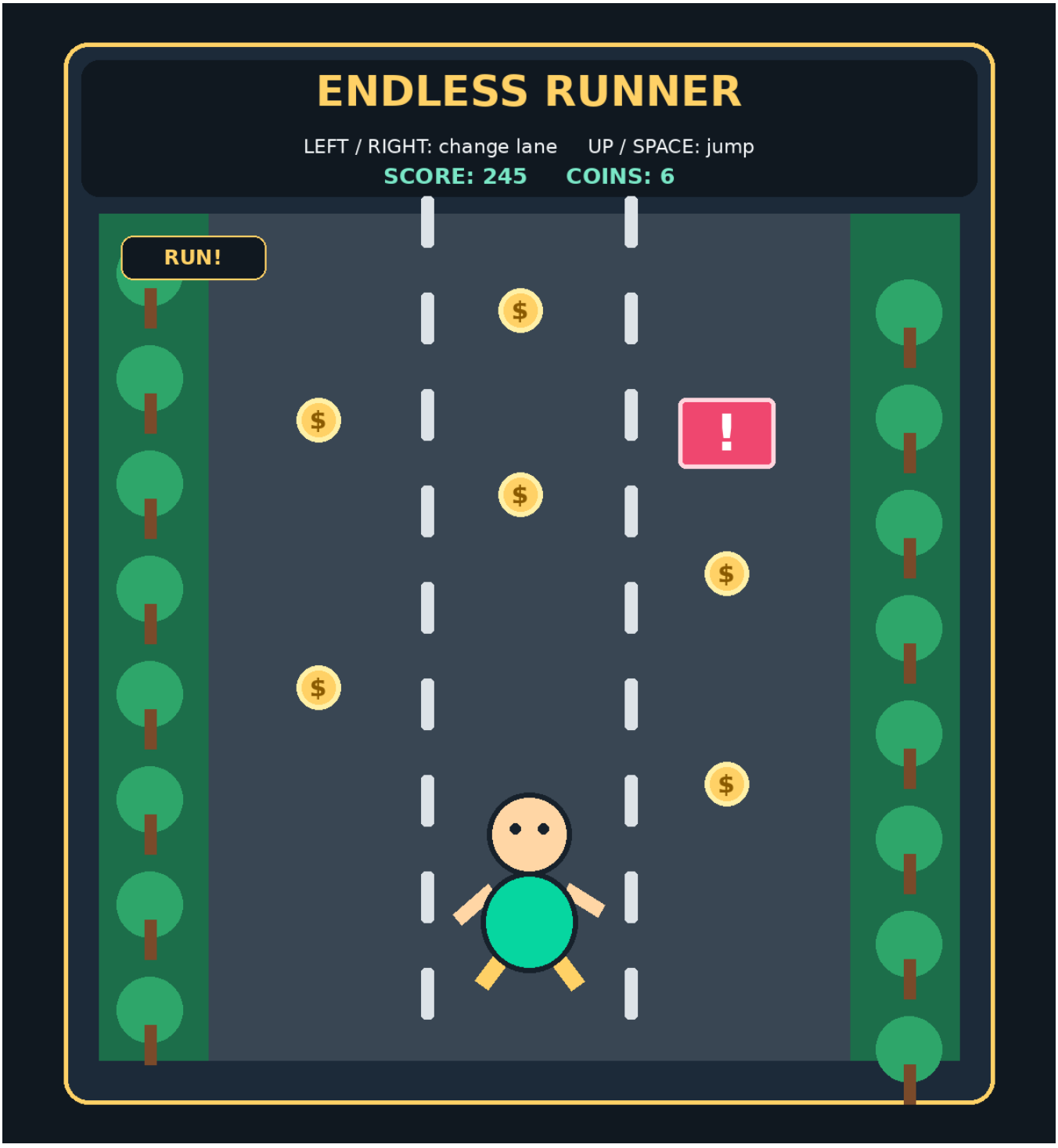
            if obj["y"] < HEIGHT + 35:
                remaining.append(obj)

        self.objects = remaining
        self.speed = min(15, self.speed + 0.003)
        self.info.config(text=f"SCORE: {self.score}   COINS: {self.coins}")
        if not self.running:
            self.draw_scene()
            self.timer = None
            return

        self.draw_scene()
        self.timer = self.root.after(TICK_MS, self.update_game)

if __name__ == "__main__":
    root = tk.Tk()
    game = EndlessRunner(root)
    root.mainloop()
```

OUTPUT — SAMPLE GAME SCREENSHOT



Illustrative sample output of the Endless Runner game. The actual game window is generated when the Python program is run.

ENDLESS RUNNER — ACTUAL GAME OUTPUT

Screens captured from a running Python Tkinter implementation

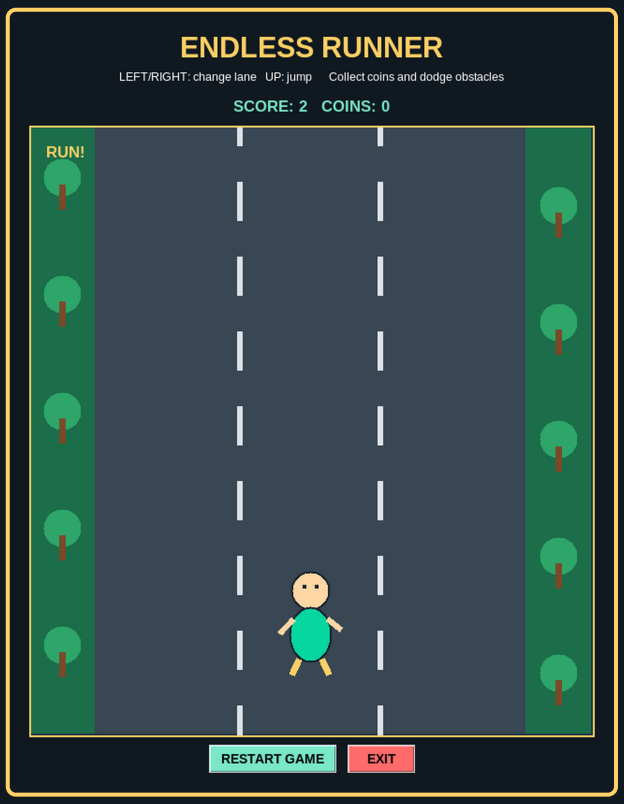


Figure 1: Initial Game Window

Actual Tkinter window showing the game title, controls, score, road, and player.

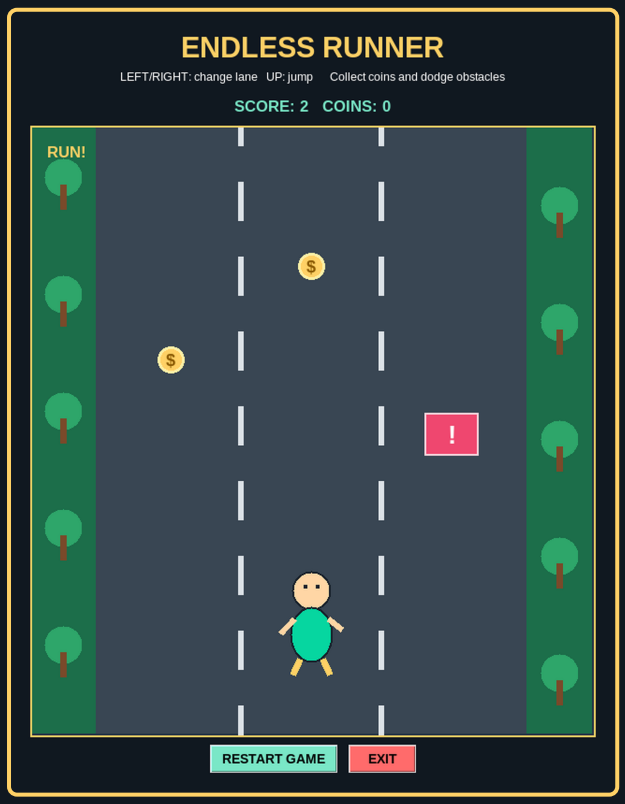


Figure 2: Starting Scene

Game scene with the player, lane markers, coins, and an obstacle.

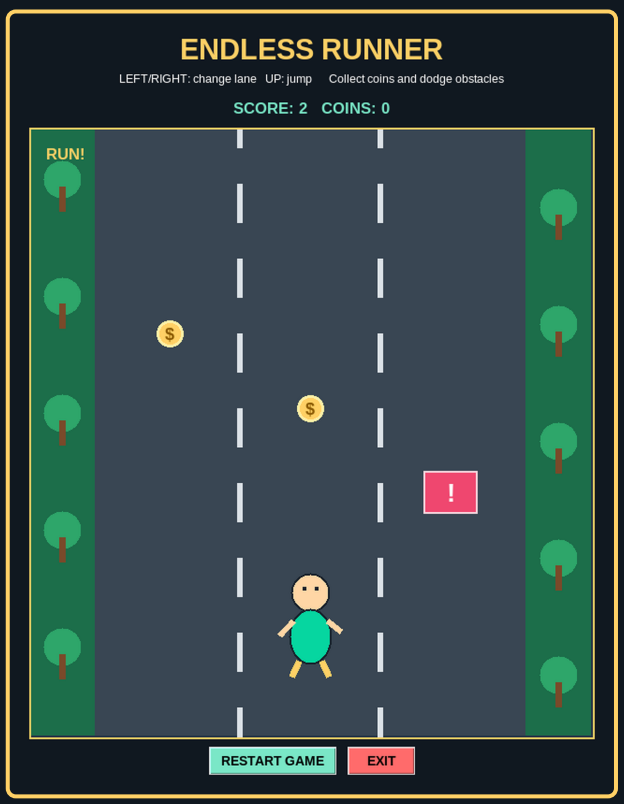


Figure 3: Gameplay

Running scene with score and coins displayed, plus obstacles to avoid.

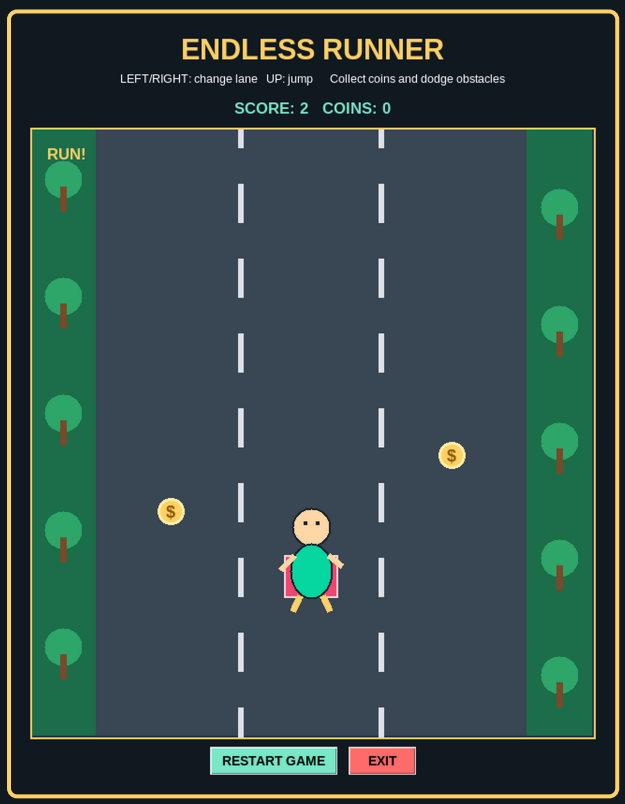


Figure 4: Jumping Over an Obstacle

Player is shown in the game's jump animation state.

ENDLESS RUNNER — ACTUAL GAME OUTPUT

Final game screen

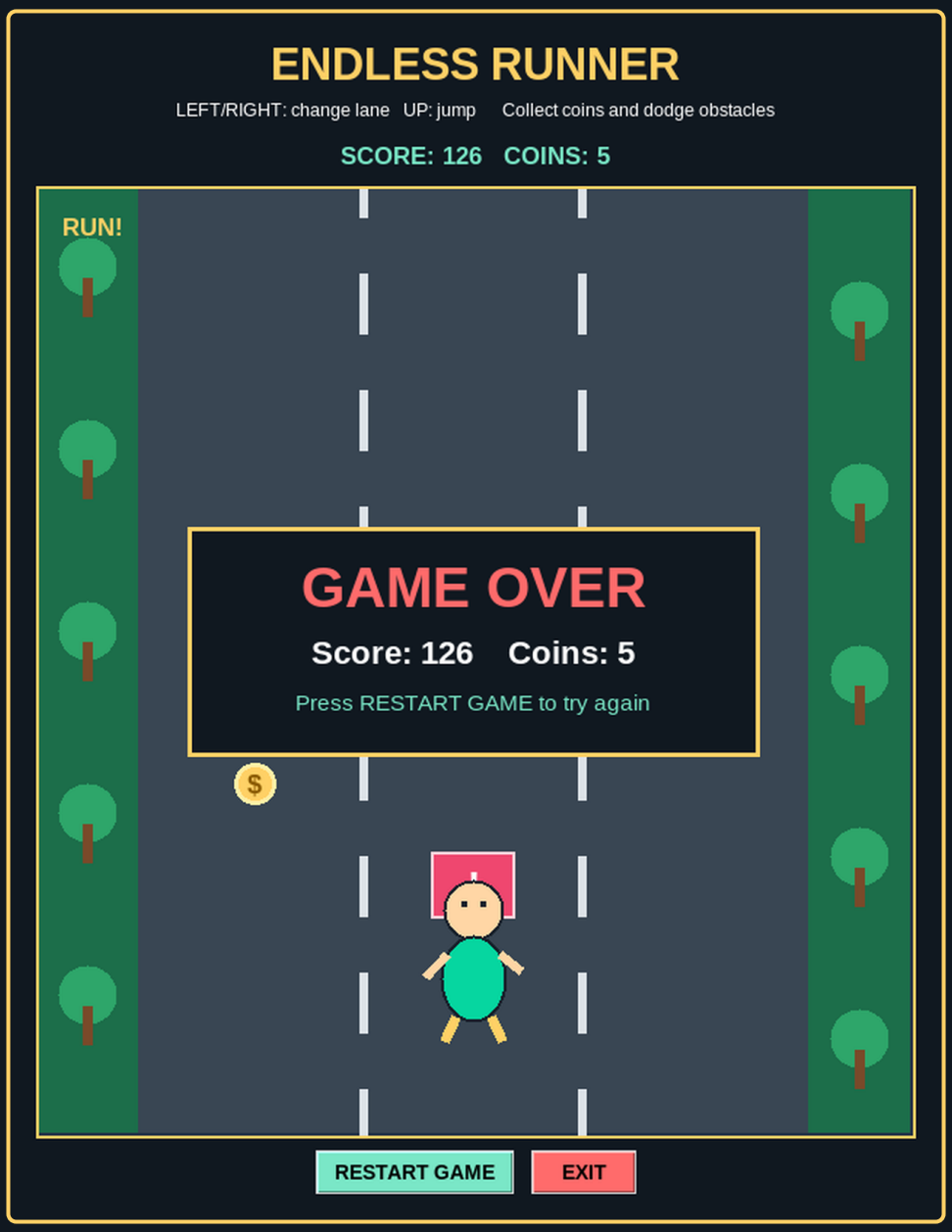


Figure 5: Final Screen — Game Over

The game's Game Over panel displays the final score and restart instruction.

OUTPUT

When executed, the game opens a window titled ENDLESS RUNNER. The player can change lanes, jump, collect coins, and avoid obstacles. The score increases while running and when coins are collected. Hitting an obstacle ends the game and shows the final score.

RESULT

Thus, an interactive 2D Endless Runner game inspired by lane-based running games was developed using Python Tkinter. The project demonstrates GUI design, keyboard event handling, animation with after(), random object generation, collision detection, scoring, and restart/exit controls.